SeeFire

Autonomous Indoor Fire Detection Robot

ASSIGNMENT 3

Module Documentation Report

Course	CSE 396 — Computer Engineering Project
University	Gebze Technical University
Department	Computer Engineering
Semester	2025–2026 Spring
Team	Group 4
Submission Date	April 2026
Document Version	v1.0

Team Members

Full Name	Student ID	Module(s) — Primary Role
Emre Can Tuncer	200104004115	M4 (AI & Vision) + M6 (Decision Engine)
Batuhan Yanar	220104004059	M7 (Data Logging) + M3 (Sensor Integration)
Ahmet Furkan Arslan	220104004044	M5 (Navigation) + M6 (Decision Engine)
Alperen Şahin	220104004943	M1 (Chassis) + M3 (Sensor Integration)
Semih Sarıkoca	220104004038	M6 (Decision Engine) + M4 (AI & Vision)
Yusuf Alperen Çelik	220104004064	M5 (Navigation) + M7 (Data Logging)
Bekir Emre Sarıpınar	220104004039	M3 (Sensor Integration) + M2 (Motor Control)
Halil Buğra Şen	230104004088	M1 (Chassis) + M6 (Decision Engine)

Table of Contents

1. Project Overview 4
2. Hardware Platform & Component List 5
3. System Architecture 7
4. Module Documentation 9
   - 4.1 M1 — Chassis & Mechanics 9
   - 4.2 M2 — Motor Control & Power 11
   - 4.3 M3 — Sensor Integration 13
   - 4.4 M4 — AI & Vision 15
   - 4.5 M5 — Navigation & Mapping 17
   - 4.6 M6 — Decision Engine (FSM) 19
   - 4.7 M7 — Data Logging & Output 21
5. Inter-Module Communication Matrix 23
6. FSM State Machine Reference 25
7. Implementation Plan 27
8. Risk Register 28

Appendix A: GPIO Pin Assignment Reference 29

Appendix B: config.py Constants 30

Appendix C: Revision History 31

1. Project Overview

1.1 Mission Statement

SeeFire is an autonomous, wheeled ground robot designed for early-stage fire and smoke detection in indoor environments (corridors, warehouses, storage rooms, server halls). The system performs full on-board inference on a single Raspberry Pi 4B — no cloud, no remote GPU — classifying it as an edge-computing robotic platform. When a fire event is confirmed with sufficient multi-sensor confidence, the robot raises a multi-modal alarm (LED + buzzer), captures a JPEG snapshot of the scene, and persists the event to a local database for post-incident forensic review.

1.2 Scope of Operation

The robot operates in three behavioural phases, governed by a finite-state machine:

- **Exploration:** When first deployed in an unknown environment, the robot performs right-hand wall-following to map the area, using two HC-SR04 ultrasonic sensors (one forward, one right-side).
- **Patrol:** Once exploration completes (robot returns near the starting point), it transitions to a patrol mode, re-traversing the mapped area at a reduced cadence.
- **Verify & Alarm:** At any point during Explore or Patrol, if the fusion score of vision + smoke + IR signals crosses a tuned threshold, the robot halts, re-examines the scene for a short window, and if confirmation holds, raises the alarm and logs the event.

1.3 Design Philosophy

The design is built on four principles:

1. **Modular Ownership:** The software is partitioned into seven modules (M1–M7), each with a clearly defined public interface, so that team members can develop in parallel without integration friction.
2. **Edge-First Inference:** All perception and decision logic runs locally. The robot produces no network traffic during normal operation, which is a critical property for deployment in security-sensitive or offline environments.
3. **Multi-Sensor Fusion:** Vision (YOLOv8n) alone generates false positives in scenes with sunlight reflections, red objects, or steam. The decision engine fuses YOLO confidence with an MQ-2 smoke sensor reading and an MLX90614 IR surface temperature reading, weighted and clamped to a single 0.0–1.0 threat score.
4. **Graceful Degradation:** Each sensor loss mode is handled explicitly — an I2C bus lockup on the MLX90614, for instance, does not crash the main loop; the module returns a status code and the fusion score uses only the remaining signals.

1.4 Relationship to Prior Deliverables

This document (Assignment 3) builds on two previous submissions:

- **Assignment 1 — Project Proposal:** Defined the problem, motivation, and high-level system requirements.
- **Assignment 2 — Module Interface & Header Design Report:** Produced the seven C-style header files that declared the public API of each module, along with an inter-module communication table and an FSM state-transition table.

The purpose of Assignment 3 is to explain *what each module does* and *how the modules connect at runtime* — a technical document that unifies the header-level contracts into an executable architectural picture. No implementation has started at the time of writing; this document describes the design that the team will execute during the upcoming Module Demonstration phase (Week 10).

2. Hardware Platform & Component List

2.1 Computing Platform

The robot is built around a single **Raspberry Pi 4 Model B (4 GB RAM)**. This specific configuration was chosen after benchmark review: the 4 GB model has sufficient memory headroom for YOLOv8n INT8 inference (~1.0–1.3 GB peak) while remaining cost-effective and widely available. The Pi 4 is thermally bounded under sustained CPU load, so the final hardware list includes an active cooling solution (aluminium heatsink + fan) to prevent throttling during continuous inference.

2.2 Final Component List

The following list is the definitive procurement list. Every component name, model, and quantity has been reviewed against each module's electrical requirements. Items marked in the rightmost column are critical integration notes that the team must observe during assembly.

#	Component	Exact Model / Specification	Qty	Integration Note
1	Main Computer	Raspberry Pi 4 Model B, 4 GB RAM	1	Runs entire software stack; GPIO/I2C/SPI active.
2	Active Cooling	Pi 4 aluminium heatsink set + 5 V cooling fan	1	Mandatory — YOLO load pushes CPU above 80 °C without cooling.
3	Storage	SanDisk 32 GB Extreme microSDHC (A1 / A2 class)	1	A1 rating required for SQLite random-write performance.
4	Camera	Logitech C270 HD (USB webcam, 720p)	1	Plug-and-play with OpenCV; no driver install needed.
5	Chassis Kit	4WD Acrylic Robot Chassis Kit (double-deck)	1	Includes 4× TT 6 V geared DC motors + 4 wheels + frame.
6	Motor Driver	L298N Dual H-Bridge Motor Driver	1	Left 2 motors wired in parallel, right 2 motors in parallel (differential drive).
7	Battery	LiPo 2S, 7.4 V, 2200 mAh, 30C, XT60 plug	1	Powers motors directly and Pi via buck converter.
8	Buck Converter	LM2596 Step-Down Module (5 V / 3 A output)	1	Supplies clean 5 V to Pi, isolating it from motor voltage dips.
9	ADC	MCP3008 SPI 8-channel 10-bit ADC (DIP-16)	1	Pi has no analog inputs. MQ-2 analog output routed through this chip.
10	Gas/Smoke Sensor	MQ-2 Gas & Smoke Sensor Module	1	Requires ~60 s warm-up before readings stabilise.
11	IR Thermometer	MLX90614 (GY-906) I2C non-contact	1	I2C wiring < 20 cm; keep reads inside try/except.
12	Ultrasonic Sensor	HC-SR04	2	Echo pin is 5 V — voltage divider mandatory for Pi GPIO (see #15).
13	Alarm LED	5 mm Red LED	2	Series with 330 Ω resistor on GPIO 26.
14	Alarm Buzzer	Active Buzzer, 5 V	1	Driven by GPIO 19 via transistor or direct for low-current modules.
15	Resistor Set	330 Ω (LED), 1 kΩ + 2 kΩ (HC-SR04 divider)	1 set	1 kΩ / 2 kΩ divider steps 5 V Echo to 3.3 V safe range for Pi GPIO.
16	Power Connector	XT60 Male Pigtail (wired connector)	1	Lets battery plug into breadboard/L298N without soldering.
17	Jumper Wires	40-pin M-F / F-F / M-M Dupont set	1 set	I2C runs kept short (< 20 cm).
18	Breadboard	400-tie-point mini breadboard	1	Power rail distribution and sensor fan-out.

CRITICAL SAFETY NOTE
The HC-SR04 Echo pin outputs a 5 V logic signal. The Raspberry Pi GPIO pins are rated for a maximum input voltage of 3.3 V. Connecting the Echo pin directly to a Pi GPIO will permanently damage the Pi. The 1 kΩ / 2 kΩ voltage divider listed above is a hardware-level requirement, not an optional best practice.

2.3 Power Distribution

Rail	Source	Voltage	Typical / Max Current	Loads
Main	LiPo 2S	7.4 V nominal (6.4–8.4 V)	— / 20 A	L298N Vcc (motor supply), LM2596 input
Logic 5 V	LM2596 output	5.0 V regulated	0.6 A / 3.0 A	Pi 4, MQ-2, Buzzer, MLX90614 Vcc (via 3.3 V LDO on module)
Pi 3.3 V	Pi on-board reg.	3.3 V	— / 0.5 A	MCP3008 Vref, HC-SR04 divider output, GPIO logic
Motor 6 V	L298N internal	~5.5 V effective	~1.5 A per side under load	4× TT 6 V motors (paralleled L+R)

With a 2S 7.4 V LiPo, the L298N's internal voltage drop of about 1.5–2 V leaves roughly 5.5 V at the motor terminals. This is close to the motor's 6 V rating — slightly underdriven, which is safer than overdriving and extends motor life without meaningful loss of torque for this chassis weight.

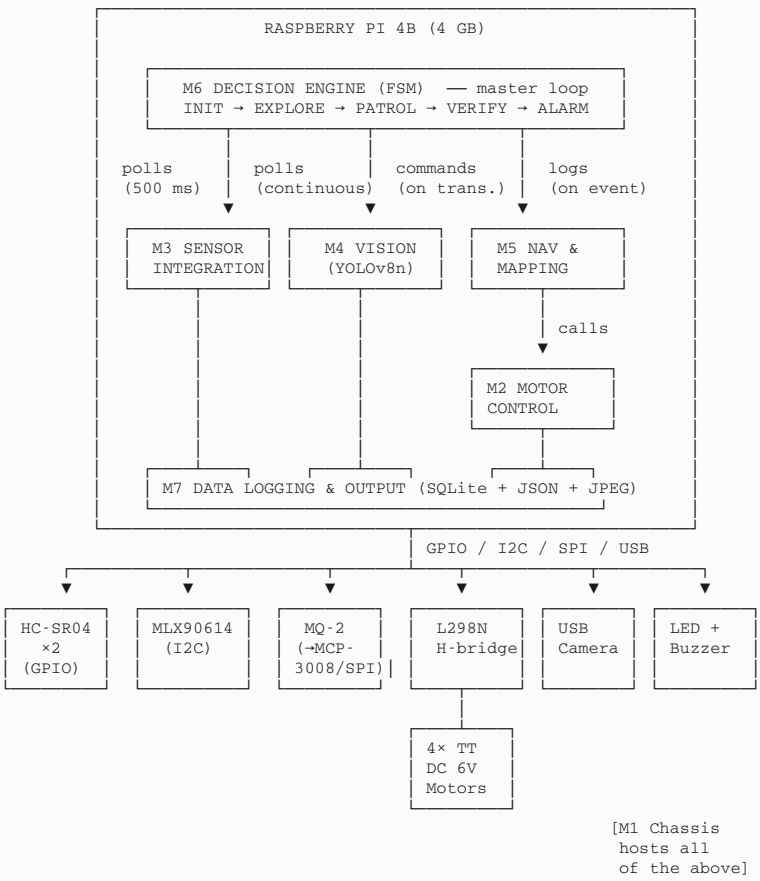
3. System Architecture

3.1 Module Map

The SeeFire software stack is partitioned into seven cooperating modules. Each module exposes a C-style public API (the header files from Assignment 2) and runs inside the same Python process, communicating via direct function calls and shared data structures. No inter-module traffic crosses a network or serial link.

ID	Module	Primary Role	Owners
M1	Chassis & Mechanics	Physical platform	Alperen Şahin, Halil Buğra Şen
M2	Motor Control & Power	Low-level actuation	Bekir Emre Sarıpınar
M3	Sensor Integration	Sensor drivers & fusion feed	Alperen Şahin, Bekir Emre Sarıpınar, Batuhan Yanar
M4	AI & Vision	YOLOv8n inference pipeline	Emre Can Tuncer, Semih Sarıkoca
M5	Navigation & Mapping	Motion planning & map persistence	Ahmet Furkan Arslan, Yusuf Alperen Çelik
M6	Decision Engine (FSM)	State machine, sensor fusion, alarm control	Emre Can Tuncer, Semih Sarıkoca, Ahmet Furkan Arslan, Halil Buğra Şen
M7	Data Logging & Output	Event DB, map & snapshot persistence	Batuhan Yanar, Yusuf Alperen Çelik

3.2 System Block Diagram



3.3 Runtime Model

The system runs as a single Python process. Internally, three cooperating threads are spawned, each owning a distinct cadence and a distinct module:

Thread	Owner Module	Cadence	Responsibility
Main / FSM	M6	~500 ms per iteration	Polls M3, M4, M5; runs state-transition logic; issues commands.
Vision	M4	~5 FPS (~200 ms)	Runs YOLOv8n inference continuously; publishes latest result to a mutex-protected struct.
Navigation	M5	~100 ms per iteration	Closed-loop motor control based on HC-SR04 readings; executes the last command issued by M6.

M2, M3, and M7 are passive libraries — they do not own threads of their own. They are called synchronously by M5, M6, or the vision thread. M4's inference thread is the reason Vision must run on its own thread: a single YOLO inference takes ~200 ms, which would block the FSM loop and delay obstacle reaction.

> **DESIGN DECISION — THREADS, NOT PROCESSES**
> An earlier draft considered separate OS processes for M4 (Vision) and M6 (FSM) to isolate GIL contention. We opted for threads because YOLOv8n inference releases the GIL during its native C++ / ONNX execution, and shared memory (one-writer / many-reader on a result struct) is significantly simpler than managing inter-process pipes. Process isolation remains a fallback if scheduling jitter proves problematic during Week-10 testing.

3.4 Coordinate & Unit Conventions

- **Distance:** centimetres (cm) at the sensor API; metres (m) at the navigation / map API.
- **Angle:** degrees, 0° = robot's forward heading at boot; clockwise positive.
- **Temperature:** degrees Celsius.
- **Time:** Unix epoch seconds (float, millisecond precision) for logging; milliseconds for loop cadences.
- **Confidence scores:** floats in [0.0, 1.0].

4. Module Documentation

Each of the following subsections describes a single module in depth: what the module does, the internal working logic that makes it do so, its public API surface (the C-style header contract), and the modules it interacts with. Headers were developed in Assignment 2 and have been reviewed and finalised for this report.

MODULE M1

Chassis & Mechanics

Physical platform — mounts, wiring, and mechanical specifications

Owners: Alperen Şahin (220104004943), Halil Buğra Şen (230104004088)

Purpose

M1 is the hardware foundation. It is the only module that is not software — it defines the mechanical frame, the motor mount geometry, the sensor mounting points, and the cable routing. Every other module depends on M1 being correctly assembled before it can be tested. Because the team cannot yet procure and assemble the chassis, the M1 "header" is a specification document: a table of dimensions, mount positions, and GPIO-to-motor mappings that freezes the physical layout so that M2–M7 can design against a stable reference.

Internal Design

The chosen platform is a double-deck acrylic 4WD kit with four 6 V TT geared DC motors. This is a 4-wheel drive configuration wired as differential drive: the two left motors share a single L298N H-bridge channel (driven in parallel), and the two right motors share the other channel. This trades independent wheel control for mechanical stability — the robot will be carrying a Raspberry Pi, battery, camera, breadboard, and multiple sensor modules, and a 2WD platform would tip under that load.

Parameter	Value
Chassis length × width × height	200 × 150 × 100 mm
Wheel diameter	65 mm
Wheelbase (front–rear)	140 mm
Track width (left–right)	120 mm
Motor type	DC 6 V geared, ~100 RPM no-load
Total motors	4 (2 left parallel, 2 right parallel)
Battery type	LiPo 2S1P, 7.4 V, 2200 mAh

Sensor Mounting Layout

Sensor	Location	Purpose
HC-SR04 (forward)	Front centre, ~50 mm height	Obstacle detection directly ahead
HC-SR04 (right side)	Right side, ~50 mm height	Wall-following reference distance
MLX90614 IR	Top-centre, ~80 mm height, tilt forward	Non-contact temperature of scene in front
MQ-2	Top-front, ~60 mm height	Air-borne smoke / combustible gases
USB Webcam	Top-front, ~90 mm height, tilt ~10° up	YOLO inference input

Public API

M1's API is minimal because it describes static hardware. It exposes two functions that let other modules query the mount configuration at runtime.

Function	Returns	Used By
`m1_get_motor_mount(motor_id)`	`M1_MotorMount` (PWM + direction pins)	M2 at initialisation
`m1_verify_assembly()`	`bool` — all mounts accessible	Integration smoke-test

Interactions With Other Modules

- **→ M2:** Provides motor pin mapping (FL=0, FR=1, RL=2, RR=3) so M2 knows which GPIO pins drive each motor.

- **→ M3:** Provides sensor mounting offsets so M3's distance readings can be translated to robot-frame coordinates by M5.
- **Depends on:** Nothing at software level — depends only on physical parts being procured and assembled.

MODULE M2

Motor Control & Power

L298N driver, PWM speed control, battery monitoring, alarm I/O

Owner: Bekir Emre Sarıpınar (220104004039)

Purpose

M2 is the only module that drives physical actuators. It owns every GPIO pin connected to the L298N motor driver, the alarm LED, and the buzzer. Every movement and every alarm event flows through M2's public API. M5 calls `m2_motor_drive()` for locomotion; M6 calls `m2_set_alarm()` when the FSM enters the ALARM state.

Internal Working Logic

The module maintains a thin GPIO abstraction layer. On `m2_init_motors()`, it configures four output pins (IN1..IN4) for direction control and two PWM channels (ENA, ENB) at a fixed 1 kHz frequency. The PWM duty cycle (0–100) is the speed parameter.

```
Direction → GPIO truth table (L298N):
  FORWARD:    IN1=H IN2=L  IN3=H IN4=L   (both sides forward)
  BACKWARD:   IN1=L IN2=H  IN3=L IN4=H   (both sides backward)
  TURN_LEFT:  IN1=L IN2=H  IN3=H IN4=L   (left back, right fwd)
  TURN_RIGHT: IN1=H IN2=L  IN3=L IN4=H   (left fwd, right back)
  STOP:       all LOW, PWM duty = 0
```

Rotation by a target angle (`m2_motor_turn`) uses timed differential drive: the turn time is calibrated once per chassis and scales linearly with the requested angle. This is a deliberately simple approach since the robot lacks a gyro — wheel-encoder odometry is not available, so M2 cannot close the loop on angle.

Battery Monitoring

The LiPo voltage (6.4–8.4 V) is read through the MCP3008 ADC via a resistor voltage divider. The divider scales the input down to < 3.3 V to stay within the ADC's reference. Two thresholds are defined:

- `M2_BATTERY_LOW_V` = 9.6 V — warning, M6 switches LED to slow blink.
- `M2_BATTERY_CRIT_V` = 8.4 V — critical, M6 halts navigation gracefully.

The header constants above were defined for a 3S pack in an earlier revision and will be re-calibrated for the final 2S 7.4 V pack during integration.

Alarm Subsystem

The LED and buzzer are driven from dedicated GPIO pins (26 and 19). M2 exposes a non-blocking blink thread so that M6 can request "fast blink" without stalling the FSM loop.

Public API (Summary)

Function	Purpose	Called By
`m2_init_motors()`	Configure GPIO/PWM	Main, once at boot
`m2_motor_drive(dir, speed)`	Move in direction at PWM speed	M5 (nav thread)
`m2_motor_stop()`	Immediate halt	M5, M6 (on ALARM)
`m2_motor_turn(angle, speed)`	Timed rotation	M5
`m2_get_battery_voltage(out)`	Read LiPo via ADC	M6 (periodically)
`m2_set_alarm(led, buzz)`	LED + buzzer on/off	M6
`m2_blink_led(pattern)`	Non-blocking blink	M6
`m2_get_state(out)`	Snapshot for diagnostics	M7 optionally
`m2_cleanup()`	Release GPIO	Main, at shutdown

Interactions With Other Modules

- ← **M5:** Receives `motor_drive` and `motor_turn` calls from the navigation loop.
- ← **M6:** Receives `set_alarm` and `blink_led` calls on state transitions; polled for battery voltage.
- **Depends on:** M1 (pin mapping), Raspberry Pi GPIO + PWM libraries.

MODULE M3

Sensor Integration

MQ-2, MLX90614, HC-SR04 ×2 drivers + unified fusion API

Owners: Alperen Şahin (220104004943), Bekir Emre Sarıpınar (220104004039), Batuhan Yanar (220104004059)

Purpose

M3 is the unified sensor hub. It initialises every sensor at boot, owns the per-sensor driver code, and exposes two consolidated data structures — one for M5 navigation and one for M6 fire detection. The rest of the stack never talks to a sensor directly; everything goes through M3. This makes sensor replacement, calibration, and fault-tolerance changes local to this module.

Internal Working Logic

On `m3_init_sensors()`, the module:

1. Opens `/dev/i2c-1` and probes the MLX90614 at address `0x5A`.
2. Opens SPI (`spidev 0.0`) for MCP3008; configures MQ-2 on channel 0.
3. Configures the four HC-SR04 GPIO pins (TRIG + ECHO per sensor) as in/out.
4. Waits ~60 s for MQ-2 warm-up before fusion readings are considered valid.

Fusion Sensor Read (for M6)

When M6 calls `m3_get_fusion_sensors()`, the module performs three reads in sequence and returns a single struct:

```
m3_fusion_data_t {
    smoke_level   ← MQ-2 raw ADC from MCP3008   (0–1023)
    smoke_alert   ← true if smoke_level > M3_SMOKE_THRESHOLD (default 300)
    ir_temp       ← MLX90614 object temp in °C
    timestamp     ← time.time() at moment of sampling
}
```

MLX90614 I2C reads are wrapped in `try/except` because the bus can lock up on long jumper wires. On failure, the module returns the previous cached reading with `M3_ERR_I2C`; the fusion score in M6 degrades gracefully instead of crashing.

Navigation Sensor Read (for M5)

`m3_get_ultrasonic_distances()` triggers both HC-SR04 sensors in sequence (not simultaneously — adjacent sensors cross-talk if triggered together) and returns distances in centimetres. Timeouts yield `-1 cm` so M5 can distinguish "out of range" from "error".

```
HC-SR04 read procedure (per sensor, ~30 ms):
  1. TRIG pin high for 10 µs then low
  2. Wait for ECHO pin to rise (timeout 50 ms)
  3. Measure ECHO high duration in µs
  4. distance_cm = duration_us / 58.0
  5. Return -1.0 if timeout or distance > 400 cm
```

Public API (Summary)

Function	Purpose	Called By
`m3_init_sensors()`	Initialise all sensors	Main, at boot
`m3_set_smoke_threshold(t)`	Override MQ-2 threshold	M6 at runtime
`m3_set_ir_temp_threshold(t_c)`	Override IR alarm temp	M6 at runtime
`m3_get_fusion_sensors(out)`	Return fire-detection sensor pack	M6 (~500 ms)
`m3_get_ultrasonic_distances(out)`	Return HC-SR04 × 2 distances	M5 (~100 ms)
`m3_get_navigation_sensors(out)`	Full nav sensor packet	M5
`m3_cleanup()`	Release GPIO/I2C/SPI	Main, at shutdown

Interactions With Other Modules

- **→ M6:** Delivers `m3_fusion_data_t` every ~500 ms (smoke + IR).
- **→ M5:** Delivers `m3_ultrasonic_t` every ~100 ms.
- **Depends on:** M1 (sensor mounts), hardware (MCP3008 on SPI, MLX90614 on I2C, HC-SR04 voltage dividers).

> **DESIGN NOTE**
> An earlier draft of the header included a `m3_get_imu_heading()` function backed by an MPU6050. After a design review, the MPU6050 was dropped because robust gyro-based heading estimation on a non-real-time Linux kernel requires calibration effort beyond the scope of this project. The function will be removed in v0.2 of the header. Heading is now inferred by M5 via timed turns (open-loop dead reckoning).

MODULE M4

AI & Vision

YOLOv8n INT8 inference pipeline on USB webcam feed

Owners: Emre Can Tuncer (200104004115), Semih Sarıkoca (220104004038)

Purpose

M4 owns the perception pipeline. It opens the USB webcam, runs a YOLOv8n model (INT8 quantised) on every captured frame, and publishes the latest detection result to a shared struct that M6 reads on every FSM iteration. It also provides a one-shot snapshot API so that M6 can capture an evidence JPEG when the alarm fires.

Internal Working Logic

On `m4_start_pipeline()` the module:

1. Loads the INT8-quantised YOLOv8n model from disk (pre-trained, fine-tuned on the Roboflow fire/smoke dataset).
2. Opens the webcam via OpenCV (`cv2.VideoCapture(0)`) at 320 × 320 resolution — the lowest resolution the model accepts, chosen to keep inference time under 200 ms per frame on a Pi 4.
3. Spawns a background thread that runs the **capture → preprocess → infer → postprocess** loop.
4. Writes the output into an `m4_vision_result_t` protected by a mutex.

```
Per-frame pipeline:

  ┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────────┐
  │  Capture │ → │ Resize & │ → │  YOLOv8n │ → │ Filter conf. │
  │  320×320 │   │ Normalise│   │  Inference│   │  > threshold │
  └──────────┘   └──────────┘   └──────────┘   └──────────────┘
                                                       │
                                                       ▼
                                                 ┌──────────────┐
                                                 │ Publish to   │
                                                 │ shared result │
                                                 │ (mutex write) │
                                                 └──────────────┘
   ~40 ms          ~15 ms         ~150 ms         ~5 ms   →  ~5 FPS target
```

Detection Result Contract

Each `m4_detection_t` carries a class ID (0 = fire, 1 = smoke), a confidence score in [0, 1], and a bounding box in the 320×320 frame. M6 consumes only the highest-confidence detection per cycle but all detections are kept in the struct for logging and for future features (e.g. tracking multiple hotspots).

Snapshot Capture

`m4_capture_snapshot()` grabs the *current* webcam frame (not the one used for the last inference), encodes it as JPEG at 75 % quality, and returns the bytes to the caller. This is only called by M6 on ALARM transitions; M7 receives the buffer and writes it to disk.

Public API (Summary)

Function	Purpose	Called By
`m4_start_pipeline()`	Load model, open camera, start thread	Main, at boot
`m4_stop_pipeline()`	Join thread, release camera	Main, at shutdown
`m4_get_latest_result(out)`	Copy latest result (thread-safe)	M6 every ~500 ms
`m4_capture_snapshot(buf, sz, len)`	One-shot JPEG capture	M6 on ALARM
`m4_is_running(out)`	Pipeline health query	M6, diagnostics

Interactions With Other Modules

- **→ M6:** Publishes `m4_vision_result_t` for fusion-score calculation.
- **→ M6 → M7:** Snapshot bytes flow M4 → M6 → M7 on alarm.
- **Depends on:** `ultralytics`, `opencv-python`, `numpy`, `Pillow`; USB webcam hardware.
- **Does NOT depend on:** M1, M2, M3, M5. M4 can be developed and tested with pre-recorded video files before the chassis is assembled — this is an important scheduling property.

MODULE M5

Navigation & Mapping

Wall-following, waypoint patrol, obstacle avoidance, map persistence

Owners: Ahmet Furkan Arslan (220104004044), Yusuf Alperen Çelik (220104004064)

Purpose

M5 turns high-level intent ("explore", "patrol", "verify heading 45°") into concrete motor commands. It owns the closed-loop navigation thread that runs at ~100 ms cadence, consuming HC-SR04 distances from M3 and emitting `motor_drive` / `motor_turn` calls to M2. It also maintains the coarse occupancy grid of explored cells and persists that grid via M7.

Internal Working Logic — Commands

M5 is effectively a small controller that reacts to commands issued by M6. The command set is:

Command	Behaviour
`M5_CMD_EXPLORE`	Right-hand wall-following. Mark each visited cell as FREE in the occupancy grid; mark obstacle-returning cells as OCCUPIED. Terminates when the robot returns close to its start pose.
`M5_CMD_PATROL`	Traverse a list of waypoints auto-generated from the explored grid. Cycles indefinitely until interrupted.
`M5_CMD_VERIFY`	Turn to a target heading and drive forward until the front ultrasonic reports an obstacle or a configured distance is covered. Used when M6 wants a closer look at a suspicious scene.
`M5_CMD_STOP`	Immediate halt. Hold position.
`M5_CMD_RESUME`	Resume whichever command was active before STOP / VERIFY.

Wall-Following Algorithm (Explore Mode)

```
Every navigation tick (~100 ms):

  left_cm, right_cm = m3_get_ultrasonic_distances()    // front, right

  IF left_cm < M5_OBSTACLE_DIST_CM:        // ≤ 20 cm in front
      m2_motor_turn(-90°, speed)           // turn left away from wall
      mark current cell OCCUPIED
  ELSE IF right_cm > M5_WALL_FOLLOW_DIST_CM + slack:
      slight right drift                   // regain wall contact
  ELSE IF right_cm < M5_WALL_FOLLOW_DIST_CM − slack:
      slight left drift                    // pull away from wall
  ELSE:
      m2_motor_drive(FORWARD, cruise_speed)
      mark current cell FREE
```

Occupancy Grid & Dead Reckoning

Position is estimated in open loop: the navigation thread integrates motor command duration and direction to update an estimated (x, y) pose. This is error-prone over long distances (drift), but acceptable for a small indoor environment — and far simpler than any alternative available without a gyro or wheel encoders. The grid resolution is 10 cm per cell, covering up to 40 × 40 cells (~4 × 4 m).

Map Persistence

On exploration completion, M5 serialises the grid to JSON and hands the string to `m7_save_map()`. On next boot, M5 calls `m7_load_map()`; if a valid grid is returned, M6 skips EXPLORE and moves directly to PATROL.

Public API (Summary)

Function	Purpose	Called By
`m5_init_navigation()`	Load saved map, start nav thread	Main, at boot
`m5_execute_command(order)`	Change active nav mode	M6 on state transition
`m5_get_nav_status(out)`	Snapshot of progress / stuck flag	M6 every iteration
`m5_update_navigation(l, r, h)`	Internal tick (testable)	Unit tests
`m5_save_map(path)`	Persist current grid	End of EXPLORE
`m5_load_map(path)`	Restore grid from disk	Boot time
`m5_cleanup()`	Stop thread, release resources	Main, at shutdown

Interactions With Other Modules

- ← **M6:** Receives `execute_command` orders; M6 polls `get_nav_status` on every FSM tick.
- → **M2:** Calls `motor_drive` / `motor_turn` / `motor_stop` every ~100 ms.
- → **M3:** Calls `get_ultrasonic_distances` every ~100 ms.
- → **M7:** Calls `save_map` / `load_map` at start and end of EXPLORE.

MODULE M6

Decision Engine (FSM)

Finite-state machine, sensor fusion scoring, alarm control

Owners: Emre Can Tuncer (200104004115), Semih Sarıkoca (220104004038), Ahmet Furkan Arslan (220104004044), Halil Buğra Şen (230104004088)

Purpose

M6 is the orchestrator. It is the only module that *decides* — everything else either senses (M3, M4), acts (M2, M5), or records (M7). M6 runs the FSM loop: poll sensors and vision, compute a fusion score, update state, dispatch commands, and log events.

Internal Working Logic — The Five States

```
                ┌──────────────┐
                │    INIT      │  boot; check /data/map.json
                └──────────────┘
       no map        │        map found
          ┌──────────┴──────────┐
          ▼                     ▼
   ┌──────────┐          ┌──────────┐
   │ EXPLORE  │◄────────►│  PATROL  │  ← reset_alarm()
   │  (M5)    │          │   (M5)   │
   └──────────┘          └──────────┘
        │     suspicious       │
        │     signal triggers  │
        ▼                      ▼
                 ┌──────────┐
                 │  VERIFY  │  fusion_score borderline
                 └──────────┘
                      │ score > 0.7
                      ▼
                 ┌──────────┐
                 │  ALARM   │  LED+buzzer, JPEG, log
                 └──────────┘
```

Fusion Score Formula

The central algorithm in M6 is the fusion score — a single 0.0–1.0 number that collapses three heterogeneous signals into one decision variable:

```
score = W_VISION · vision_conf
      + W_SMOKE  · (smoke_level / 1023)
      + W_IR     · clamp((ir_temp − 40) / 60, 0, 1)

with W_VISION = 0.5, W_SMOKE = 0.3, W_IR = 0.2   (sum = 1.0)
```

These weights bias the score toward vision (which has the best spatial localisation) but keep meaningful weight on smoke and IR so that a sunlit red surface alone (vision_conf ≈ 0.6) cannot cross the alarm threshold of 0.7.

State-Transition Rules

From	To	Condition
INIT	EXPLORE	`m7_load_map()` returns error / empty
INIT	PATROL	`m7_load_map()` returns valid grid
EXPLORE / PATROL	VERIFY	vision_conf > 0.5 OR smoke_alert OR ir_temp > 55 °C
VERIFY	ALARM	fusion_score > 0.7
VERIFY	PATROL / EXPLORE	fusion_score < 0.4 (false alarm, logged as such)
ALARM	EXPLORE / PATROL	Manual reset OR configurable timeout

Main Loop (pseudocode)

```
while running:
    state   = get_state()
    sensors = m3_get_fusion_sensors()          // smoke + IR
    vision  = m4_get_latest_result()
    nav     = m5_get_nav_status()
    battery = m2_get_battery_voltage()

    score   = calculate_fusion_score(vision, sensors)

    new_state = fsm_transition(state, vision, sensors, score, nav)

    if new_state != state:
        m7_log_event(STATE_CHANGE, …)
        if new_state == ALARM:
            jpeg = m4_capture_snapshot()
            m7_save_snapshot(jpeg)
            m2_set_alarm(True, True)
            m7_log_event(FIRE_DETECTED, …)

        m5_execute_command(cmd_for(new_state))

    sleep(M6_SENSOR_POLL_MS)                   // 500 ms
```

Public API (Summary)

Function	Purpose	Called By
`m6_init()`	Wire dependencies, enter INIT	Main
`m6_run()`	Blocking FSM loop	Main
`m6_stop()`	Signal loop to exit	Signal handler
`m6_get_state(out)`	Current FSM state	Diagnostics / M7
`m6_calculate_fusion_score(v, s, out)`	Compute 0–1 score	Internal / tests
`m6_trigger_alarm(event)`	Dispatch alarm to M2 + M7	Internal
`m6_reset_alarm()`	Return to EXPLORE / PATROL	Manual reset

Interactions With Other Modules

- → **M3:** Polls `get_fusion_sensors` every 500 ms.
- → **M4:** Polls `get_latest_result` every 500 ms; calls `capture_snapshot` on ALARM.
- → **M5:** Issues `execute_command`; polls `get_nav_status`.
- → **M2:** Calls `set_alarm`, `blink_led`, polls battery.
- → **M7:** Calls `log_event` on every state change and alarm event.

MODULE M7

Data Logging & Output

SQLite event log, JSON map persistence, JPEG snapshot storage

Owners: Batuhan Yanar (220104004059), Yusuf Alperen Çelik (220104004064)

Purpose

M7 is the persistence layer. It owns every write to the SD card: the SQLite event database (`/data/seefire.db`), the occupancy-grid JSON (`/data/map.json`), and the alarm JPEG snapshots (`/data/snapshots/`). Because it has no dependencies on other modules, M7 can be developed and tested first — the rest of the stack then uses it as a reliable substrate.

Internal Working Logic

On `m7_init_database()` the module opens (or creates) the SQLite file and issues the table DDL:

```
CREATE TABLE IF NOT EXISTS events (
    row_id       INTEGER PRIMARY KEY AUTOINCREMENT,
    event_type   INTEGER NOT NULL,
    timestamp    REAL    NOT NULL,
    pos_x        REAL,
    pos_y        REAL,
    confidence   REAL,
    vision_conf  REAL,
    smoke_level  INTEGER,
    ir_temp      REAL,
    snapshot_path TEXT,
    details      TEXT
);
```

Event Types

ID	Event	Emitted By
0	STATE_CHANGE	M6 on every FSM transition
1	FIRE_DETECTED	M6 on ALARM entry
2	FALSE_ALARM	M6 when VERIFY → PATROL (score dropped)
3	EXPLORATION_DONE	M5 on full loop closure
4	PATROL_STARTED	M6 on PATROL entry
5	OBSTACLE_AVOIDED	M5 when wall-follow turn fires
6	LOW_BATTERY	M6 when V < M2_BATTERY_LOW_V
7	SYSTEM_START	Main at boot
8	SYSTEM_STOP	Main at graceful shutdown

Snapshot Storage

JPEG snapshots are written as `img_NNN.jpg` in the snapshots directory, where `NNN` is a zero-padded incrementing counter scoped to the current DB. The absolute path is returned to M6 and stored in the event row, so every `FIRE_DETECTED` row points to its evidence image.

Map Persistence

`m7_save_map()` writes a caller-provided JSON string atomically (write-to-tmp then rename) so a crash mid-write cannot corrupt the map. `m7_load_map()` reads the file into a caller-owned buffer; a missing file returns `M7_ERR_READ`, which M5 interprets as "no prior map, enter EXPLORE".

Public API (Summary)

Function	Purpose	Called By
`m7_init_database()`	Open DB, create tables	Main, very early
`m7_log_event(evt, rowid)`	Insert event row	M6 on every transition
`m7_get_events(type, limit, out, n)`	Query recent events	Post-mortem / debug
`m7_close_database()`	Graceful SQLite close	Main, at shutdown
`m7_save_map(json, path)`	Atomically write map.json	M5 end-of-EXPLORE
`m7_load_map(buf, size, path)`	Read map.json to buffer	M5 at boot
`m7_save_snapshot(data, sz, path_out)`	Write JPEG, auto-increment name	M6 on ALARM

Interactions With Other Modules

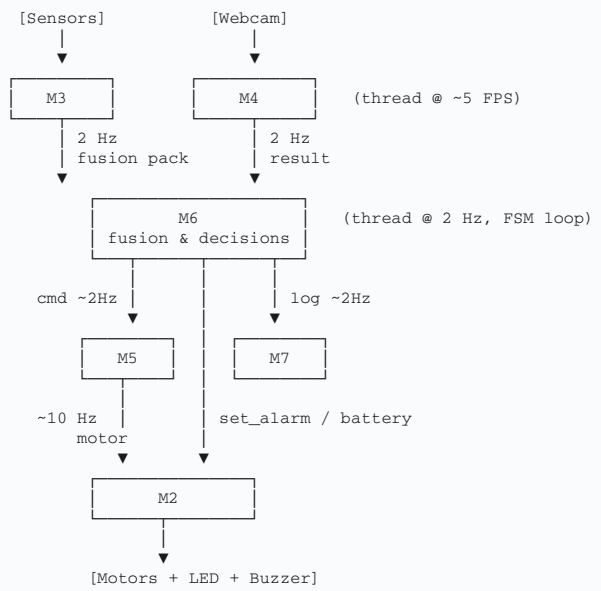
- **← M6:** Receives event rows on every state change + alarm JPEGs.
- **← M5:** Receives map save / load requests.
- **Depends on:** SD card, Python `sqlite3` stdlib, `json` stdlib. No module dependencies — M7 is a leaf.

5. Inter-Module Communication Matrix

The following table enumerates every cross-module call. Every row is a runtime dependency: if the target module is unavailable or faulty, the caller must handle the failure (return code or exception). Rates are approximate.

IF#	Caller	Callee	Function	Payload	Rate
IF-01	M5	M2	`m2_motor_drive / stop / turn`	direction, speed	~10 Hz
IF-02	M6	M2	`m2_set_alarm / blink_led`	bool flags / pattern	On transition
IF-03	M6	M2	`m2_get_battery_voltage`	float out	~1 Hz
IF-04	M5	M3	`m3_get_ultrasonic_distances`	`m3_ultrasonic_t`	~10 Hz
IF-05	M6	M3	`m3_get_fusion_sensors`	`m3_fusion_data_t`	~2 Hz
IF-06	M6	M4	`m4_get_latest_result`	`m4_vision_result_t`	~2 Hz
IF-07	M6	M4	`m4_capture_snapshot`	JPEG bytes	On ALARM
IF-08	M6	M5	`m5_execute_command`	`m5_nav_order_t`	On transition
IF-09	M6	M5	`m5_get_nav_status`	`m5_nav_status_t`	~2 Hz
IF-10	M5	M7	`m7_save_map / load_map`	JSON string	On event
IF-11	M6	M7	`m7_log_event`	`m7_event_t`	~2 Hz (peaks)
IF-12	M6	M7	`m7_save_snapshot`	JPEG bytes	On ALARM
IF-13	Main	M*	`*_init / *_cleanup`	—	Boot / shutdown

5.1 Data-Flow Diagram — Happy Path



5.2 Alarm Path — Event Flow

```
TRIGGER:  fusion_score > 0.7 inside M6 VERIFY state

  M6 ──► m4_capture_snapshot()         (~5 ms)
       ─► m7_save_snapshot(jpeg)       (~15 ms)
       ─► m2_set_alarm(True, True)     (~1 ms)
       ─► m7_log_event(FIRE_DETECTED)  (~3 ms)
       ─► m5_execute_command(STOP)     (~1 ms)

Result: LED on, buzzer on, JPEG on SD, event row in DB, robot halted.
Elapsed: < 50 ms from threshold crossing to locked-down state.
```

6. FSM State Machine Reference

Every runtime behaviour of SeeFire collapses into one of five discrete states owned by M6. The state is updated on each FSM tick (~500 ms). The complete transition table, the actions executed on each transition, and the events logged by M7 are enumerated below.

6.1 State Summary

State	ID	What the Robot Is Doing	Nav Command Issued
INIT	0	Hardware init, map lookup, sensor warm-up	STOP
EXPLORE	1	Right-hand wall-following, mapping cells	EXPLORE
PATROL	2	Cycling through saved waypoints	PATROL
VERIFY	3	Approaching a suspicious heading; fusion score borderline	VERIFY
ALARM	4	LED + buzzer on, snapshot taken, event logged	STOP

6.2 Complete Transition Table

From	To	Trigger	Actions
INIT	EXPLORE	No valid map on disk	m5_execute(EXPLORE); log STATE_CHANGE
INIT	PATROL	Valid map loaded by M5	m5_execute(PATROL); log STATE_CHANGE + PATROL_STARTED
EXPLORE	PATROL	nav_status.exploration_complete == true	m5_save_map(); m5_execute(PATROL); log EXPLORATION_DONE
EXPLORE / PATROL	VERIFY	vision_conf > 0.5 OR smoke_alert OR ir_temp > 55 °C	m5_execute(VERIFY, heading); log STATE_CHANGE
VERIFY	ALARM	fusion_score > 0.7	m4_capture_snapshot; m7_save_snapshot; m2_set_alarm(on); m7_log FIRE_DETECTED
VERIFY	PATROL / EXPLORE	fusion_score < 0.4 (signal dropped — false alarm)	m5_execute(RESUME); log FALSE_ALARM
ALARM	PATROL / EXPLORE	Manual reset OR timeout expired	m2_set_alarm(off); m5_execute(RESUME); log STATE_CHANGE
Any	STOP (soft)	Battery < M2_BATTERY_CRIT_V	m2_motor_stop; log LOW_BATTERY; m2_blink_led(SLOW)

6.3 Fusion Score Worked Example

Suppose YOLO reports fire at confidence 0.80, the MQ-2 reads 500/1023, and the MLX90614 sees 75 °C. The fusion score evaluates as:

```
score = 0.5 · 0.80                        = 0.400
      + 0.3 · (500 / 1023)                = 0.147
      + 0.2 · clamp((75 − 40) / 60, 0, 1) = 0.117
                                            ─────
                                            0.664

0.664 < 0.7  →  remain in VERIFY, do not alarm.
```

If smoke rose to 800/1023 on the next cycle:

```
score = 0.400 + 0.3 · (800/1023) + 0.117 = 0.751  → ALARM
```

7. Implementation Plan

No code exists at the time of writing; all seven modules are still at the header-and-specification stage. The following plan orders implementation so that dependencies are always ready before dependants, and so that hardware-dependent modules (M2, M3, M5) do not block progress on hardware-independent modules (M4, M7).

7.1 Phase-Based Schedule

Phase	Week	Modules	Parallel Tracks	Milestone
A	Wk 8	M7	Track 1: SQLite schema, JSON persistence, JPEG write. No hardware needed.	M7 unit-tested on dev laptop.
B	Wk 8–9	M4 + M1	Track 1: M4 with recorded fire video clips. Track 2: M1 chassis procurement & assembly.	YOLOv8n streams detections at ≥ 3 FPS; chassis mechanically complete.
C	Wk 9	M2 + M3	Track 1: M2 motor drive on chassis. Track 2: M3 sensor drivers on breadboard.	Robot moves manually; sensors return readings.
D	Wk 9–10	M5	Wall-following + obstacle avoidance closed-loop on real chassis.	Robot completes a single lap of a test corridor unassisted.
E	Wk 10	M6	FSM + fusion scoring; ties all previous modules together.	End-to-end demo: explore → detect → verify → alarm → log.

7.2 Dependency Graph

```
M7 ──────────────────────────────────┐
  └──► M5 ──► M6                     │
          ▲   ▲                      │
M1 ──► M2 ─┤   │                     │
       └──► M5─┘                     │
M1 ──► M3 ────► M5                   │
             └► M6                   │
M4 (standalone) ──► M6               │
                                     ▼
                            [Full system run]
```

7.3 Early Development Enablers

- **Recorded video:** M4 can be unit-tested on YouTube fire clips before the camera is even connected. This lets the vision team work in parallel with the hardware team.
- **Mock M3:** A stub `m3_get_fusion_sensors` that returns pre-scripted sequences allows M6's FSM logic to be tested with zero physical sensors.
- **Mock M5 status:** A stub `m5_get_nav_status` lets M6 verify transition logic without any navigation hardware.
- **M7 first:** Because M7 has no dependencies, implementing it first gives every later module a reliable logging target from day one.

8. Risk Register

Each risk was identified during design review, rated by likelihood (L) and impact (I) on a 1–3 scale (1 = low, 3 = high), and assigned a primary mitigation. The table is ordered by L × I score.

ID	Risk	L	I	Score	Mitigation
R1	YOLOv8n inference too slow on Pi 4 → FSM loop starves obstacle avoidance.	3	3	9	INT8 quantisation; 320×320 input; vision in dedicated thread; reduce inference FPS target to 3 if needed.
R2	GPIO pin damage from 5 V HC-SR04 Echo signal.	2	3	6	1 kΩ / 2 kΩ voltage divider on every Echo pin; verified during bring-up before first power-on.
R3	LiPo voltage sag under motor stall → Pi brown-out.	2	3	6	LM2596 buck converter isolates Pi from motor rail; add 1000 µF bulk capacitor on 5 V rail.
R4	MLX90614 I2C bus lockup on long jumpers → fusion score loses IR component.	2	2	4	Keep I2C leads < 20 cm; wrap reads in try/except; return cached last value on error; warn in log.
R5	False positives from sunlight / red objects → unnecessary ALARM.	3	2	6	VERIFY state requires fusion score > 0.7 (not vision alone); smoke + IR weights damp vision bias.
R6	Dead-reckoning drift → wall-following loop never closes → robot never enters PATROL.	2	2	4	Exploration timeout (configurable); fall back to PATROL on timeout and log event.
R7	Late component delivery (Turkey supplier lead time).	2	3	6	Procurement order issued immediately after Assignment 3 submission; all items sourced locally (Istanbul).
R8	SD-card corruption from abrupt power loss (LiPo fully discharged).	1	3	3	Atomic write (tmp + rename) for map.json; SQLite WAL mode planned for v0.2; enforce M2_BATTERY_CRIT_V soft-stop.

Appendix A: GPIO Pin Assignment Reference

This is the single source of truth for GPIO / SPI / I2C / USB wiring. Any code that references a physical pin *must* import the symbol from `config.py`; literal pin numbers are prohibited.

A.1 GPIO Pins (BCM Numbering)

Function	GPIO	Header Pin	Direction	Owner Module
L298N IN1 (left motors dir A)	17	11	OUT	M2
L298N IN2 (left motors dir B)	18	12	OUT	M2
L298N IN3 (right motors dir A)	27	13	OUT	M2
L298N IN4 (right motors dir B)	22	15	OUT	M2
L298N ENA (left PWM)	12	32	PWM	M2
L298N ENB (right PWM)	13	33	PWM	M2
HC-SR04 TRIG (front)	23	16	OUT	M3
HC-SR04 ECHO (front, via divider)	24	18	IN	M3
HC-SR04 TRIG (right)	25	22	OUT	M3
HC-SR04 ECHO (right, via divider)	8	24	IN	M3
MCP3008 CS	5	29	OUT (SPI)	M3
Alarm LED	26	37	OUT	M2
Alarm Buzzer	19	35	OUT	M2

A.2 SPI (for MCP3008 ADC)

Signal	GPIO	Header Pin
MOSI	10	19
MISO	9	21
SCLK	11	23
CE0	8 or 5	24 / 29

A.3 I2C (for MLX90614)

Signal	GPIO	Header Pin
SDA	2	3
SCL	3	5

A.4 USB

Port	Device
USB 2.0 (top)	Logitech C270 Webcam

A.5 Voltage Divider Schematic (HC-SR04 Echo → Pi GPIO)

```
      HC-SR04 Echo (5 V) ──┬── 1 kΩ ──┬── Pi GPIO (3.3 V safe)
                           │          │
                           ┴          2 kΩ
                                      │
                                      ┴ GND

Vout = 5 V · 2 / (1 + 2) = 3.33 V  →  within Pi GPIO spec
```

Appendix B: config.py Shared Constants

Every tunable constant referenced by the seven modules lives in a single `config.py` file at the project root. Each module imports the constants it needs — literals inside module source are forbidden.

```
# config.py — SeeFire global configuration

# ────── GPIO / BCM numbering ──────────────────────────────────────
MOTOR_IN1, MOTOR_IN2, MOTOR_IN3, MOTOR_IN4 = 17, 18, 27, 22
MOTOR_ENA, MOTOR_ENB                       = 12, 13
TRIG_FRONT, ECHO_FRONT                     = 23, 24
TRIG_RIGHT, ECHO_RIGHT                     = 25,  8
MCP3008_CS                                 =  5
LED_PIN                                    = 26
BUZZER_PIN                                 = 19

# ────── I2C addresses ─────────────────────────────────────────────
I2C_BUS            = 1      # /dev/i2c-1
MLX90614_ADDR      = 0x5A

# ────── SPI ───────────────────────────────────────────────────────
SPI_BUS            = 0
SPI_DEVICE         = 0
MQ2_ADC_CHANNEL    = 0      # MCP3008 channel 0

# ────── Detection thresholds ──────────────────────────────────────
VISION_CONF_THRESH = 0.5    # enter VERIFY
SMOKE_THRESHOLD    = 300    # MQ-2 raw ADC (0..1023)
IR_TEMP_THRESHOLD  = 55.0   # °C; enter VERIFY

# ────── Fusion weights (must sum to 1.0) ──────────────────────────
W_VISION           = 0.5
W_SMOKE            = 0.3
W_IR               = 0.2
FUSION_ALARM_THRESH = 0.7
FUSION_CLEAR_THRESH = 0.4

# ────── Battery monitoring (2S LiPo, 7.4 V nominal) ───────────────
BATTERY_LOW_V      = 7.0    # warning  (3.5 V / cell)
BATTERY_CRIT_V     = 6.6    # soft-stop (3.3 V / cell)

# ────── Navigation ────────────────────────────────────────────────
OBSTACLE_DIST_CM       = 20
WALL_FOLLOW_DIST_CM    = 30
NAV_LOOP_MS            = 100
CRUISE_SPEED           = 60     # PWM duty (0..100)
GRID_RESOLUTION_M      = 0.10
GRID_WIDTH_CELLS       = 40
GRID_HEIGHT_CELLS      = 40

# ────── FSM / Vision pipeline ─────────────────────────────────────
SENSOR_POLL_MS         = 500
VISION_INPUT_WIDTH     = 320
VISION_INPUT_HEIGHT    = 320
VISION_FPS_TARGET      = 5

# ────── Paths ─────────────────────────────────────────────────────
DATA_DIR               = "/data"
DB_PATH                = "/data/seefire.db"
MAP_PATH               = "/data/map.json"
SNAPSHOT_DIR           = "/data/snapshots/"
```

CALIBRATION NOTE
The battery thresholds shown are for the final 2S (7.4 V) LiPo pack and override the 3S values that appear in an earlier draft of the M2 header. Actual values will be fine-tuned on the bench once the buck converter and pack are wired together.

Appendix C: Revision History

C.1 Document Revisions

Version	Date	Authors	Changes
v1.0	April 2026	SeeFire Team (Group 4)	Initial Assignment 3 release. Covers seven modules, inter-module matrix, FSM reference, implementation plan, and full risk register.

C.2 Key Changes Since Assignment 2

- **Ownership correction:** M6 team now correctly lists Emre Can Tuncer as one of the owners (moved from M2).
- **Hardware finalised:** MPU6050 and DHT22 removed. HC-SR04 voltage divider made a hard requirement.
- **Battery pack:** Switched from a 3S-implied design to a confirmed 2S 7.4 V LiPo; battery thresholds re-calibrated.
- **Cooling:** Active heatsink + fan added to the component list to prevent thermal throttling under YOLO load.
- **M3 API:** `m3_get_imu_heading` is slated for removal in v0.2 of the header (IMU dropped).
- **Chassis:** 4WD configuration retained for stability under payload; wiring plan is "2 motors per L298N channel, paralleled".

C.3 Planned Header Revisions (v0.2)

Module	Change	Reason
M2	Update `M2_BATTERY_LOW_V` / `M2_BATTERY_CRIT_V` to 2S values	Final battery confirmed as 2S 7.4 V
M3	Remove `m3_get_imu_heading`; remove `m3_nav_data_t` IMU field	MPU6050 dropped from BOM
M5	Remove `imu_heading_deg` argument from `m5_update_navigation`	Consistent with M3 change
M6	Add configurable alarm-reset timeout constant	Currently hard-coded

END OF DOCUMENT

SeeFire — Group 4 — CSE 396 — Spring 2025-2026